

Badania operacyjne

Informatyka 2025/2026

Skład zespołu:

Imię i Nazwisko

Jan Martowicz

Grzegorz Grudzień

Oliwia Zawila

Maciej Borowiec

Spis treści

1. Wstęp	3
2. Opis zagadnienia	3
2.1. Sformułowanie problemu	3
2.2. Model matematyczny	3
3. Opis algorytmów	4
3.1. Faza 1: Symulowane Wyżarzanie (Optymalizacja częstotliwości)	4
3.1.1. Reprezentacja rozwiązania	4
3.1.2. Wygenerowanie rozwiązania początkowego	4
3.1.3. Dokładny opis procedur (Sąsiedztwo, chłodzenie)	4
3.1.4. Pseudokod	5
3.2. Faza 2: Ciągłe mikro-harmonogramowanie (Dystrybuanta CDF)	5
3.2.1. Reprezentacja rozwiązania	5
3.2.2. Dokładny opis procedur (Mapowanie na oś ciągłą)	6
4. Aplikacja	7
4.1. Instrukcja uruchomienia (Krok po kroku)	7
4.2. Charakterystyka danych wejściowych	7
4.3. Konfiguracja hiperparametrów	8
4.4. Wyniki i generowanie plakatów PDF	8
5. Eksperymenty	8
5.1. Metodologia i środowisko testowe	9
5.2. Strojenie hiperparametrów Symulowanego Wyżarzania	9
5.2.1. Temperatura początkowa (<code>initial_temp</code>)	9
5.2.2. Temperatura minimalna (<code>min_temp</code>)	10
5.2.3. Współczynnik chłodzenia (<code>cooling_rate</code>)	10
5.2.4. Przeszukiwanie siatkowe (Grid Search)	11
5.3. Analiza wrażliwości parametrów modelu	12
5.3.1. Korelacje parametrów z metrykami wyjściowymi	12
5.3.2. Mapy cieplne rozkładu kursów w ciągu doby	13
5.3.3. Rozkłady liczby kursów — wykresy skrzypcowe	14
6. Podsumowanie	16
7. Dodatek – podział pracy	17

1. Wstęp

Projekt przedstawia aplikację wspierającą decyzje dyspozytorskie w zakresie optymalizacji rozkładu jazdy tramwajów. System realizuje dwufazowy proces tworzenia harmonogramu, którego celem jest znalezienie optymalnego kompromisu pomiędzy komfortem pasażerów a kosztami operacyjnymi przewoźnika. Wykorzystując zaawansowane metody heurystyczne oraz ciągle mikro-harmonogramowanie, system przetwarza surowe dane o popycie i czasach przejazdów, aby ostatecznie wygenerować płynny, gotowy do druku rozkład jazdy. Aplikacja demonstruje praktyczne zastosowanie algorytmów optymalizacyjnych w zarządzaniu miejskim transportem zbiorowym.

2. Opis zagadnienia

Zarządzanie komunikacją miejską wymaga ciągłego dostosowywania podaży (liczby kursów) do zmieniającego się w ciągu doby popytu pasażerskiego. Zbyt mała liczba tramwajów prowadzi do przepełnienia pojazdów i długiego czasu oczekiwania na przystankach, z kolei nadmierna liczba kursów generuje wysokie koszty operacyjne i wymaga utrzymywania niepotrzebnie dużej floty pojazdów. Dodatkowym wyzwaniem jest płynne łączenie okresów o różnym natężeniu ruchu (np. przejście ze szczytu porannego w międzyszczyt), tak aby uniknąć niefizycznych luk i szokowych zmian w częstotliwości kursowania.

2.1. Sformułowanie problemu

Problem polega na wyznaczeniu optymalnego harmonogramu odjazdów tramwajów w oparciu o wejściowe dane przestrzenno-czasowe. Należy zdefiniować:

- optymalną liczbę kursów w każdej godzinie doby (Faza 1).
- konkretne, dokładne co do minuty czasy odjazdów (Faza 2).

System musi uwzględniać ograniczenia infrastrukturalne (maksymalna przepustowość torowisk) oraz parametry taboru (pojemność tramwaju). Rozwiązanie powinno minimalizować globalną funkcję kosztu, która waży ze sobą dwa sprzeczne interesy: czas oczekiwania pasażerów oraz zapotrzebowanie na tabor.

2.2. Model matematyczny

Problem można sformułować jako zadanie optymalizacji dyskretniej, w którym zmienną decyzyjną jest wektor częstotliwości kursowania w poszczególnych godzinach.

Niech:

- P – macierz popytu (liczba pasażerów między węzłami w czasie).
- W – macierz czasów przejazdu między węzłami.
- C – pojemność pojedynczego pojazdu.
- R – czas obrotu tramwaju (pełny kurs w obie strony w minutach).
- x_h – liczba kursów w godzinie h .
- V_h – wielkość niezbędnej floty pojazdów w danej godzinie, wyliczana wprost ze wzoru: $V_h = \lceil x_h \cdot \frac{R}{60} \rceil$.

Funkcja celu Z (do minimalizacji) składa się z dwóch ważonych komponentów:

$$Z = \alpha \cdot \text{CałkowityCzasOczekiwania} + \beta \cdot \text{CałkowitaFlota}$$

Gdzie:

- α – waga priorytetu pasażera (kara za całkowity czas oczekiwania w systemie).
- β – waga priorytetu kosztowego (kara za zaangażowanie floty operacyjnej).

Dodatkowo model musi podlegać ograniczeniu przepustowości $x_h \leq \text{max_frequency}$ dla każdego h .

3. Opis algorytmów

3.1. Faza 1: Symulowane Wyżarzanie (Optymalizacja częstotliwości)

Do wyznaczenia optymalnej liczby kursów dla każdej z 24 godzin zastosowano metaheurystykę Symulowanego Wyżarzania (Simulated Annealing). Algorytm ten, inspirowany procesem stygnięcia metali, pozwala na unikanie utknięcia w minimach lokalnych poprzez akceptację – z pewnym malejącym prawdopodobieństwem – rozwiązań gorszych.

3.1.1. Reprezentacja rozwiązania

Rozwiązanie jest reprezentowane jako jednowymiarowa tablica (wektor) liczb całkowitych o długości 24. Każdy indeks odpowiada jednej godzinie doby, a wartość pod tym indeksem określa liczbę zaplanowanych kursów w tej godzinie (np. [4, 4, 8, 12, 15, ...]).

3.1.2. Wygenerowanie rozwiązania początkowego

Rozwiązanie początkowe w algorytmie jest całkowicie deterministyczne i opiera się na wstępnej estymacji zapotrzebowania. System sumuje cały popyt dla danej godziny ze wszystkich relacji, a następnie wyznacza minimalną liczbę kursów niezbędną do obsłużenia tego popytu, zakładając idealne wypełnienie pojazdu. Gwarantowane jest uruchomienie co najmniej jednego kursu w każdej godzinie. Formalnie, dla każdej godziny h , częstotliwość początkowa to:

$$x_h = \max(1, \lceil \text{popyt}_h / C \rceil)$$

3.1.3. Dokładny opis procedur (Sąsiedztwo, chłodzenie)

- **Generowanie sąsiedztwa:** Nowe rozwiązanie (sąsiad) powstaje poprzez drobną modyfikację aktualnego wektora częstotliwości. Algorytm losowo wybiera jedną godzinę, a następnie zmienia zaplanowaną na nią liczbę kursów o wartość Δ wylosowaną ze zdefiniowanego zbioru dopuszczalnych zmian: $\{-2, -1, 1, 2\}$. Po dodaniu Δ , nowa liczba kursów jest rygorystycznie przycinana do przedziału dopuszczalnego $[0, \text{max_frequency}]$.
- **Procedura chłodzenia i akceptacji:** Algorytm w każdej z maksymalnie max_iter iteracji wylicza różnicę kosztów ΔZ między nowym a obecnym rozwiązaniem. Zmiany poprawiające rozwiązanie ($\Delta Z < 0$) są akceptowane zawsze. Zmiany pogarszające są akceptowane z prawdopodobieństwem $\exp(-\frac{\Delta Z}{T})$. Na koniec każdej iteracji temperatura jest mnożona przez stały współczynnik cooling_rate ($T_{\text{nowa}} = T_{\text{stara}} \cdot \text{cooling_rate}$). Niezależnie od bieżącego stanu algorytmu (który może się pogorszyć na drodze eksploracji), system globalnie zapisuje i aktualizuje historycznie najlepsze napotkane rozwiązanie.

3.1.4. Pseudokod

```
ZAINICJALIZUJ wektor f_obecne (wg wzoru: max(1, ceil(popyt_h / C)))
ZAINICJALIZUJ T = initial_temp
OBLICZ energia_obecna = FunkcjaCelu(f_obecne, alfa, beta, P, C, R)

f_najlepsze = f_obecne
energia_najlepsza = energia_obecna

DLA krok OD 1 DO max_iter:
    JEŚLI T < min_temp:
        PRZERWIJ PĘTLĘ

    WYBIERZ losowy indeks godziny 'idx' w f_obecne
    WYLOSUJ delta ze zbioru {-2, -1, 1, 2}

    UTWÓRZ f_nowe = kopia(f_obecne)
    f_nowe[idx] = MAX(0, MIN(f_nowe[idx] + delta, max_frequency))

    OBLICZ energia_nowa = FunkcjaCelu(f_nowe, alfa, beta, P, C, R)
    delta_energii = energia_nowa - energia_obecna

    JEŚLI delta_energii < 0 LUB LOSUJ(0, 1) < exp(-delta_energii / T):
        f_obecne = f_nowe
        energia_obecna = energia_nowa

    JEŚLI energia_obecna < energia_najlepsza:
        energia_najlepsza = energia_obecna
        f_najlepsze = kopia(f_obecne)

    T = T * cooling_rate

ZWRÓĆ f_najlepsze
```

3.2. Faza 2: Ciągłe mikro-harmonogramowanie (Dystrybuanta CDF)

Faza druga modelu odpowiada za transformację makroskopowych decyzji planistycznych (optymalnych częstotliwości godzinowych $best_f$ wyznaczonych w Fazie 1) w mikroskopowy, minutowy rozkład jazdy.

Głównym problemem klasycznego, równomiernego przypisywania odjazdów wewnątrz odrębnych godzin (np. dzielenia 60 minut przez liczbę kursów f_t) jest powstawanie нефizycznych luk oraz gwałtownych skoków częstotliwości na granicach okien czasowych (tzw. problem schodka). Przykładowo, przy spadku częstotliwości z 6 do 2 kursów na godzinę, na styku przedziałów pasażerowie mogą napotkać niespodziewaną, 30-minutową przerwę.

Aby wyeliminować to zjawisko, zaimplementowany w module `timetable_phase.py` algorytm traktuje całą dobę jako jedną ciągłą oś czasu i dokonuje dyskretnej aproksymacji ciągłej dystrybuanty (CDF – Cumulative Distribution Function) łącznej liczby kursów. Przejścia między godzinami o różnym popycie ulegają w ten sposób naturalnemu wygładzeniu, a interwały międzykursowe zmieniają się płynnie.

3.2.1. Reprezentacja rozwiązania

W procesie mikro-harmonogramowania wyróżnia się trzy poziomy reprezentacji danych:

1. **Dane wejściowe (Wejście Fazy 2):** Wektor optymalnych częstotliwości godzinowych $F = [f_0, f_1, \dots, f_{\{23\}}]$, gdzie $f_t \in \mathbb{N}$ oznacza zaplanowaną liczbę kursów w godzinie t , oraz macierz czasów przejazdu W .
2. **Reprezentacja ciągła (Przestrzeń pośrednia):** Ciągła oś skumulowanej liczby podróży, na której każdy fizyczny kurs reprezentowany jest przez globalny, liczbocentryczny indeks $k \in \{0, 1, \dots, \text{total_trips} - 1\}$.
3. **Tabela relacyjna (Wyjście Fazy 2):** Ostateczna struktura danych w postaci obiektu DataFrame. Rozkład reprezentowany jest jako zbiór rekordów (wierszy), gdzie każdy wiersz jednoznacznie określa zdarzenie ruchowe poprzez zestaw cech: $S = \angle \text{TripID}, \text{Stop}, \text{DepartureTime_Min}, \text{ClockTime}, \text{HourWindow} \angle$

3.2.2. Dokładny opis procedur (Mapowanie na oś ciągłą)

Procedura wyznaczania minut odjazdów zaimplementowana w funkcji `generate_timetable` przebiega w następujących krokach matematyczno-algorytmicznych:

1. **Obliczanie skumulowanych przesunięć trasy (Offsets):** Na podstawie macierzy wag przejść W wyznaczany jest wektor skumulowanych czasów dotarcia do kolejnych przystanków względem pętli początkowej: $O_i = \sum_{j=0}^{i-1} W_{\{j,j+1\}}$ dla $i \in \{1, \dots, \text{num_stops} - 1\}$, $O_0 = 0.0$
2. **Konstrukcja dyskretnej dystrybuanty (CDF):** Algorytm buduje wektor mapowania godzinowego `cumulative_trips_at_hour`. Każdy element tego wektora definiuje łączną (skumulowaną) liczbę tramwajów, które powinny opuścić pętlę od początku doby do końca godziny t : $\text{CDF}(t) = \sum_{i=0}^{t-1} f_i$, $\text{CDF}(0) = 0.0$
3. **Wyznaczanie środków ciężkości kursów (Koncepcja $k + 0.5$):** Dla każdego globalnego kursu k obliczana jest wartość celownicza (tzw. `target`): $\text{target} = k + 0.5$. Dodanie ułamka 0.5 reprezentuje geometryczny środek ciężkości interwału obsługiwanego przez dany pojazd. Zapobiega to asymetrycznemu przesunięciu odjazdów do krawędzi przedziałów czasowych i gwarantuje bufor ochronny na początku oraz na końcu doby.
4. **Lokalizacja okna czasowego i interpolacja liniowa:** Dla wyznaczonej wartości `target` algorytm przeszukuje wektor skumulowany w poszukiwaniu takiego indeksu godziny t , który spełnia warunek: $\text{CDF}(t) \leq \text{target} < \text{CDF}(t + 1)$. Po znalezieniu właściwego przedziału, względna pozycja kursu wewnątrz tej konkretnej godziny obliczana jest za pomocą ułamka interpolacyjnego: $\text{fraction} = \frac{\text{target} - \text{CDF}(t)}{f_t}$. Wartość `fraction` $\in [0, 1)$ określa, w jakiej części danej godziny powinien nastąpić odjazd.
5. **Denormalizacja czasu i zaokrąglenie:** Względny ułamek zostaje przeliczony na absolutną minutę doby (`exact_min`), a następnie zrzutowany na przestrzeń liczb całkowitych za pomocą zaokrąglenia matematycznego do najbliższej pełnej minuty: $\text{exact_min} = (t \cdot 60) + (\text{fraction} \cdot 60)$, $\text{start_min} = \text{round}(\text{exact_min})$
6. **Propagacja czasu na trasie i formatowanie:** Dla każdego zidentyfikowanego momentu startu z pętli (`start_min`), wewnętrzna pętla dokonuje liniowej propagacji czasu dla wszystkich kolejnych przystanków na trasie: $t_{\text{arrival}} = \text{start_min} + O_i$. Otrzymana minuta doby poddawana jest arytmetyce zegarowej w celu wyznaczenia czytelnej dla pasażera formatu dwukropkowego (HH:MM): $H = \left\lfloor \frac{t_{\text{arrival}}}{60} \right\rfloor \bmod 24$, $M = t_{\text{arrival}} \bmod 60$. Zastosowanie operacji $\bmod 24$ stanowi bezpiecznik dla kursów nocnych realizowanych na przełomie dób.

4. Aplikacja

Głównym interfejsem sterującym oraz środowiskiem uruchomieniowym systemu jest interaktywny notatnik Jupyter Notebook (`app.ipynb`). Projekt został zaprojektowany zgodnie z paradygmatem modularyzacji – cała zaawansowana logika obliczeniowa oraz operacje I/O zostały wydzielone do zewnętrznych skryptów języka Python (`src/`), podczas gdy notatnik pełni rolę przejrzystego panelu kontrolnego (Dashboardu) dla dyspozytora.

4.1. Instrukcja uruchomienia (Krok po kroku)

Uruchomienie kompletnego procesu optymalizacji i generowania rozkładów jazdy z perspektywy użytkownika przebiega w sposób sekwencyjny i zautomatyzowany:

1. **Przygotowanie środowiska:** Przed uruchomieniem aplikacji należy zainstalować wymagane pakiety programistyczne oraz biblioteki systemowe niezbędne do renderowania plików PDF. Instalację zależności Python realizuje się za pomocą menedżera pakietów komendą:

```
pip install -r requirements.txt
```
2. **Wprowadzenie danych:** Użytkownik umieszcza pliki wejściowe w formacie `.csv` w dedykowanym katalogu strukturalnym `data/input/`.
3. **Inicjalizacja i konfiguracja:** Po otwarciu pliku `app.ipynb` w środowisku Jupyter, użytkownik uruchamia pierwsze komórki, które odpowiadają za wygenerowanie instancji testowych (w przypadku braku danych realnych) oraz załadowanie danych do struktur macierzowych. W kolejnym kroku definiowane są warianty parametrów algorytmu.
4. **Wykonanie obliczeń:** Użytkownik uruchamia komórki sekcji optymalizacyjnej (Faza 1) oraz wygładzania matematycznego (Faza 2). Program na bieżąco raportuje postępy spadku energii w konsoli.
5. **Odbiór wyników:** Ostatnia komórka automatycznie eksportuje ostateczny harmonogram do bazy danych, po czym generuje zestaw spersonalizowanych plakatów przystankowych w formacie PDF, które zapisywane są bezpośrednio w folderze `data/output/`.

4.2. Charakterystyka danych wejściowych

System operuje na trzech plikach tekstowych rozdzielanych przecinkami (CSV), które użytkownik poddaje edycji przed uruchomieniem optymalizacji. Moduł `data_loader.py` odczytuje te zasoby i mapuje je na wysokowydajne struktury macierzowe biblioteki NumPy:

- **Plik `travel_times.csv`:** Definiuje topologię sieci i czasy przejazdów między sąsiednimi krawędziami grafu. Zawiera kolumny `Origin`, `Destination` oraz `TravelTime_min`. Zostaje wczytany jako dwuwymiarowa macierz kwadratowa wag przejazdu $W[i, j]$, gdzie indeksy odpowiadają numerom przystanków.
- **Plik `demand.csv`:** Reprezentuje prognozowane potoki pasażerskie w skali całej doby. Zawiera kolumny `Origin_s`, `Destination_d`, `TimeWindow_t` (przedział godzinowy od 1 do 24) oraz `Demand_P`. Zostaje przekształcony w trójwymiarowy tensor popytu $P[s, d, t]$.
- **Plik `stop_names.csv`:** Służy do humanizacji danych technicznych. Mapuje identyfikatory numeryczne przystanków na ich realne nazwy miejskie za pomocą kolumn `stop_id` oraz `stop_name`. Funkcja `load_stop_names` weryfikuje poprawność tego pliku pod kątem zgodności z wymiarem macierzy popytu.

4.3. Konfiguracja hiperparametrów

W celu zapewnienia pełnej kontroli nad przebiegiem procesu optymalizacji, aplikacja udostępnia scentralizowany blok konfiguracyjny. Dzieli się on na parametry operacyjne algorytmu oraz parametry fizyczno-ekonomiczne modelu:

Zmienna	Opis funkcjonalny	Wartość domyślna
initial_temp	Temperatura początkowa odpowiadająca za poziom chaosu i eksplorację na starcie.	5.0
min_temp	Temperatura minimalna wyznaczająca stan zamrożenia układu (warunek stopu).	0.001
cooling_rate	Współczynnik chłodzenia determinujący tempo geometrycznego spadku temperatury.	0.999
max_iter	Maksymalna dopuszczalna liczba iteracji (bezpiecznik pętli wyżarzania).	1000
max_frequency	Limit przepustowości infrastruktury – maksymalna liczba kursów na godzinę.	30
C	Fizyczna pojemność pasażerska pojedynczego wagonu tramwajowego.	40
R	Czas pełnego cyklu/obrotu taboru na linii w minutach.	30
ALPHA	Waga priorytetu minimalizacji czasu oczekiwania podróży w funkcji celu.	1.0
BETA	Waga priorytetu kosztowego – ograniczenie wielkości floty pojazdów.	30.0

4.4. Wyniki i generowanie plakatów PDF

Po zakończeniu Fazy 1 (wyznaczenie wektora optymalnych częstotliwości godzinowych) oraz Fazy 2 (płynne ułożenie minut za pomocą dystrybucji CDF), ostateczny rozkład zostaje zapisany w formie tabelarycznej.

Za końcowy etap prezentacji odpowiada dedykowany moduł `pdf_generator.py`. Funkcja `generate_pdf_posters` pobiera wygenerowaną ścieżkę do pliku `final_timetable.csv` oraz słownik nazw pobrany z bazy danych przystanków.

Moduł ten dynamicznie grupuje odjazdy w bloki godzinowe i buduje strukturę dokumentu HTML z zaawansowanymi stylami paged-media CSS. Silnik renderujący `WeasyPrint` kompiluje te pliki w profesjonalne, sformatowane w standardzie A4 rozkłady jazdy – osobno dla każdego przystanku. Na każdym plakacie automatycznie wyróżniana jest aktualna pozycja przystanku na trasie oraz kierunek końcowy, co czyni dokument gotowym do bezpośredniego wydruku i ekspozycji na przystankach miejskich.

5. Eksperymenty

W celu weryfikacji poprawności implementacji oraz oceny wrażliwości algorytmu na parametry konfiguracyjne przeprowadzono serię eksperymentów numerycznych. Badania obejmują strojenie hiperparametrów Symulowanego Wyżarzania (notatnik `experiments.ipynb`), analizę wrażliwości parametrów modelu (skrypt `tester.py`) oraz wizualizacje w katalogu `charts/`.

5.1. Metodologia i środowisko testowe

Eksperymenty przeprowadzono w środowisku Python 3 z wykorzystaniem bibliotek NumPy, Pandas oraz Matplotlib. Jako dane testowe posłużyły syntetyczne instancje problemu generowane modulem `data_generator.py` — linia z 5 przystankami i 24 oknami czasowymi, z profilem popytu odwzorowującym typowy rytm miejski (szczyty poranne i popołudniowe, redukcja nocna).

Główna analiza wrażliwości parametrów modelu została zrealizowana skryptem `tester.py`. Dla macierzy korelacji uruchomiono pełną siatkę $5 \times 5 \times 5 \times 5 = 625$ kombinacji parametrów (C , R , α , β), każdą po 5 razy z uśrednieniem wyników. Wykresy heatmap i violin przedstawiają natomiast wariantowanie pojedynczego parametru przy pozostałych ustalonych na wartościach bazowych ($C = 30$, $R = 15$, $\alpha = 0.1$, $\beta = 5.0$). Każda konfiguracja w tych testach również była powtarzana 5-krotnie. Pełny pipeline obejmował Fazę 1 (Symulowane Wyżarzanie) oraz Fazę 2 (generowanie rozkładu minutowego). Jako metrykę wyjściową rejestrowano liczbę kursów w poszczególnych godzinach doby.

Przetestowane zakresy parametrów:

Parametr	Symbol	Zakres testowany
Pojemność taboru	C	[30, 40, 50, 75, 100] pasażerów
Czas obrotu	R	[15, 30, 45, 60, 90] minut
Waga czasu oczekiwania	α	[0.1, 1.0, 5.0, 10.0, 20.0]
Waga kosztu floty	β	[5.0, 10.0, 50.0, 75.0, 100.0]

Parametry algorytmu wyżarzania utrzymano na stałym poziomie produkcyjnym: `initial_temp = 10.0`, `min_temp = 10-3`, `cooling_rate = 0.999`, `max_iter = 1000`, `max_frequency = 30`.

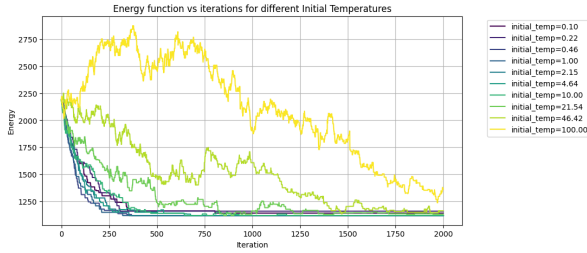
Uzupełniając, w notatniku `experiments.ipynb` przeprowadzono strojenie hiperparametrów SA na instancji 24-godzinnej ($C = 40$, $R = 30$, $\alpha = 1.0$, $\beta = 30.0$, `max_iter = 2000`) oraz — w celu walidacji skalowalności — na instancji tygodniowej (168 okien czasowych, `max_iter = 5000`). Parametry odniesienia algorytmu wyżarzania: `initial_temp = 10.0`, `min_temp = 10-3`, `cooling_rate = 0.999`, `log_every = 1`.

5.2. Strojenie hiperparametrów Symulowanego Wyżarzania

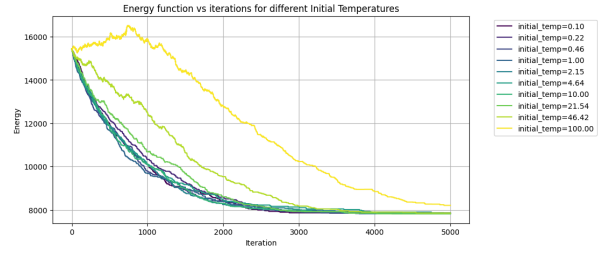
W tej serii eksperymentów wariantowano pojedynczy hiperparametr algorytmu SA, utrzymując pozostałe na wartościach bazowych. Jako metrykę jakości przyjęto wartość funkcji celu Z (energii) w funkcji numeru iteracji. Poniższe wykresy przedstawiają krzywe zbieżności — para obok siebie zestawia wyniki dla doby (24 h) i tygodnia (168 h).

5.2.1. Temperatura początkowa (`initial_temp`)

Przetestowano 10 wartości rozłożonych logarytmicznie w przedziale $[0.1, 100]$ (`np.logspace(-1, 2, 10)`).



Rysunek 1: Instancja 24-godzinna.

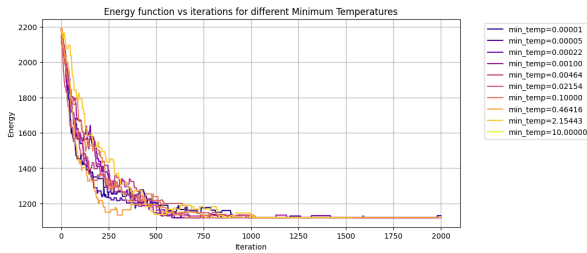


Rysunek 2: Instancja tygodniowa (168 h).

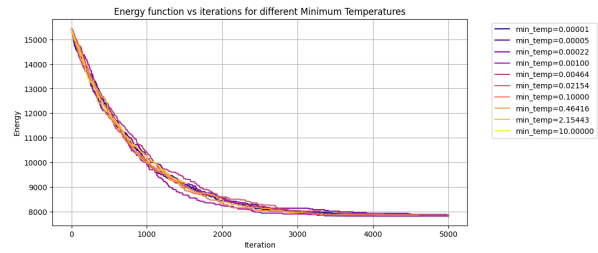
Niskie temperatury początkowe (0.1–4.6) zapewniają szybką zbieżność do minimum energii ($Z \approx 1120$ na dobie) w ciągu pierwszych 250–400 iteracji, lecz ograniczają eksplorację przestrzeni rozwiązań. Średnie wartości (1.0–21.5) osiągają ten sam poziom końcowy przy nieco większej zmienności na początku przebiegu. Bardzo wysoka temperatura początkowa (100.0) powoduje początkowy wzrost energii (eksploracja gorszych stanów) i wolniejszą zbieżność — na dobie po 2000 iteracjach energia pozostaje wyraźnie powyżej optimum ($Z \approx 1240$). Na instancji tygodniowej wszystkie krzywe startują od wyższego poziomu energii (≈ 22000), lecz wartości ≤ 46 zbiegają do $Z \approx 7800$; temperatura 100.0 nie osiąga minimum w limicie 5000 iteracji.

5.2.2. Temperatura minimalna (`min_temp`)

Przetestowano 10 wartości logarytmicznych z przedziału $[10^{-5}, 10]$.



Rysunek 3: Instancja 24-godzinna.

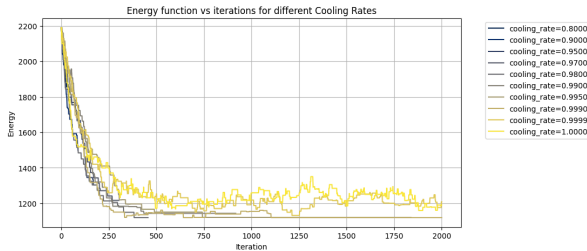


Rysunek 4: Instancja tygodniowa (168 h).

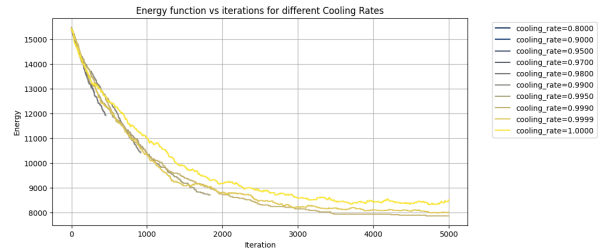
Parametr `min_temp` okazuje się najmniej wpływowy spośród testowanych hiperparametrów. Na instancji 24-godzinnej wszystkie wartości z przedziału $[10^{-5}, 2.15]$ prowadzą do tego samego minimum energii ($Z = 1119$) w okolicach iteracji 1000. Dopiero `min_temp = 10.0` (przedwczesne zatrzymanie chłodzenia) degraduje wynik do $Z = 2190$. Na instancji tygodniowej analogiczny wzorec występuje przy `min_temp` ≥ 2.15 , gdzie energia końcowa gwałtownie rośnie (do 15450 przy wartości 10.0), podczas gdy niższe progi zbiegają do $Z \approx 7810$.

5.2.3. Współczynnik chłodzenia (`cooling_rate`)

Przetestowano 10 wartości: 0.80, 0.90, 0.95, 0.97, 0.98, 0.99, 0.995, 0.999, 0.9999 oraz 1.0.



Rysunek 5: Instancja 24-godzinna.



Rysunek 6: Instancja tygodniowa (168 h).

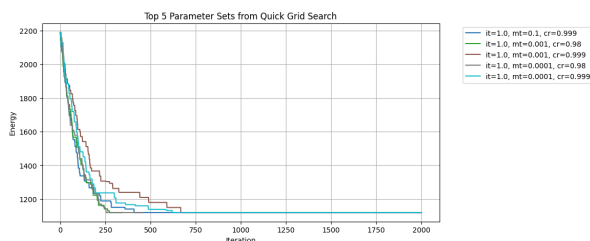
Współczynnik chłodzenia ma największy wpływ na jakość końcowego rozwiązania. Poniższa tabela zestawia energię końcową dla instancji 24-godzinnej:

cooling_rate	Końcowa energia Z
0.800	1827
0.900	1498
0.950	1303
0.970	1216
0.980	1119
0.990	1143
0.995	1120
0.999	1119
0.9999	1131
1.000	1159

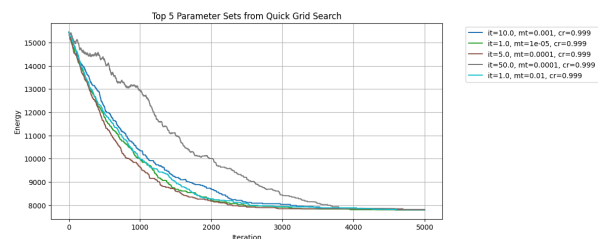
Przy szybkim chłodzeniu (≤ 0.97) krzywe spadają gwałtownie, lecz „zamrażają się”, na wysokim poziomie energii — algorytm utyka w minimach lokalnych. Optymalna strefa to $[0.98, 0.999]$ z minimum $Z = 1119$. Wartość 1.0 (brak chłodzenia) utrzymuje dużą zmienność energii i gorszy wynik końcowy. Na instancji tygodniowej różnice są jeszcze wyraźniejsze: $\text{cooling_rate} = 0.999$ daje $Z = 7858$, podczas gdy 0.80 — 14945. Wolne chłodzenie (0.999–1.0) charakteryzuje się dużą fluktuacją energii przez cały przebieg optymalizacji.

5.2.4. Przeszukiwanie siatkowe (Grid Search)

Przeprowadzono pełne przeszukiwanie siatkowe 125 kombinacji trzech hiperparametrów: $\text{initial_temp} \in \{1, 2, 5, 10, 50\}$, $\text{min_temp} \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$, $\text{cooling_rate} \in \{0.90, 0.95, 0.98, 0.99, 0.999\}$. Poniżej przedstawiono krzywe zbieżności pięciu najlepszych zestawów.



Rysunek 7: Instancja 24-godzinna — top 5 zestawów parametrów.



Rysunek 8: Instancja tygodniowa (168 h) — top 5 zestawów parametrów.

Na instancji 24-godzinnej wszystkie pięć najlepszych kombinacji osiąga identyczną energię końcową $Z = 1119$. Zestawy z $\text{cooling_rate} = 0.98$ zbiegają najszybciej (ok. 250–300 iteracji), natomiast konfiguracje z $\text{cooling_rate} = 0.999$ wymagają 400–700 iteracji. Różnice w min_temp mają drugorzędny wpływ na tempo zbieżności przy wysokim współczynniku chłodzenia. Na instancji tygodniowej najlepszy zestaw to $\text{initial_temp} = 10.0$, $\text{min_temp} = 0.001$, $\text{cooling_rate} = 0.999$ ($Z = 7794$); zestawy z szybszym chłodzeniem ($\text{cr} = 0.98$) zbiegają szybciej, lecz kończą na nieco wyższej energii.

Wnioski ze strojenia SA:

- cooling_rate w przedziale 0.98–0.999 jest parametrem o największym wpływie na jakość rozwiązania — wartość domyślna 0.999 w aplikacji jest uzasadniona.
- min_temp poniżej 10^{-2} nie wpływa istotnie na wynik końcowy, o ile nie jest zbyt wysoki.

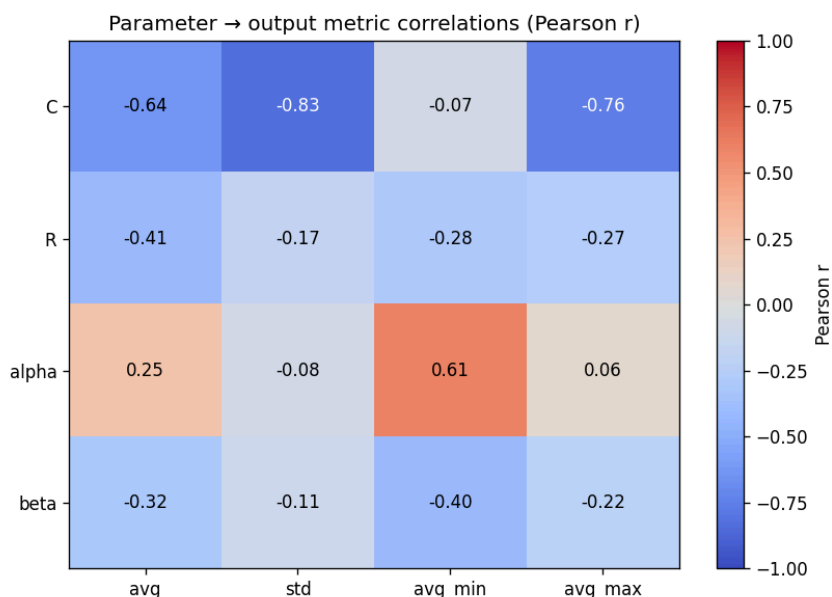
- `initial_temp` w zakresie 0.5–20 zapewnia dobrą równowagę między eksploracją a zbieżnością; ekstremalnie wysokie wartości wymagają więcej iteracji.
- Trendy obserwowane na dobie utrzymują się na instancji tygodniowej, choć absolutne wartości energii są wyższe, a zbieżność wymaga większej liczby iteracji.

5.3. Analiza wrażliwości parametrów modelu

Poniższa analiza obejmuje zarówno parametry fizyczne (C , R), jak i wagi ekonomiczne funkcji celu (α , β). Wykresy korelacji i rozrzutu uśrednionych wyników z 5 powtórzeń każdej konfiguracji pozwalają ocenić, w jaki sposób poszczególne dźwignie decyzyjne wpływają na końcowy rozkład liczby kursów w ciągu doby.

5.3.1. Korelacje parametrów z metrykami wyjściowymi

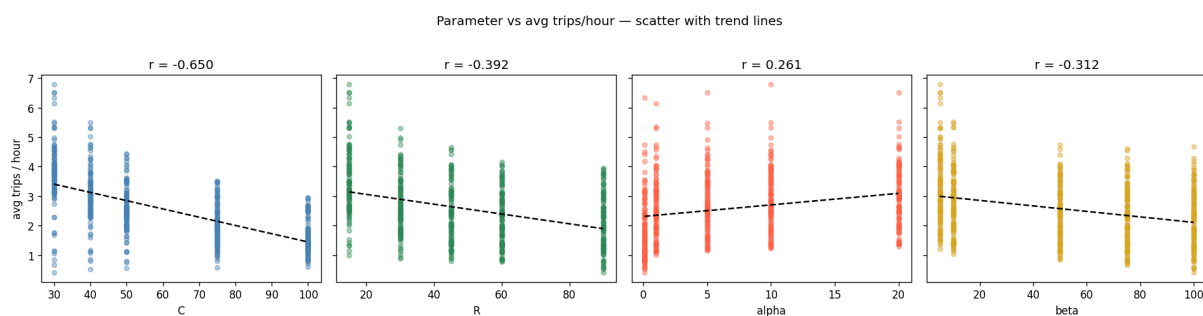
Na podstawie pełnej siatki wyników obliczono współczynniki korelacji Pearsona między parametrami wejściowymi a czterema metrykami opisującymi rozkład kursów: średnią godzinową (`avg`), odchyleniem standardowym (`std`), średnim minimum (`avg_min`) oraz średnim maksimum (`avg_max`) liczby kursów na godzinę.



Rysunek 9: Macierz korelacji Pearsona: parametry wejściowe vs. metryki rozkładu kursów.

Macierz korelacji ujawnia wyraźne, intuicyjne zależności. Parametr C (pojemność pojazdu) wykazuje silną korelację ujemną ze wszystkimi metrykami liczby kursów ($r = -0.64$ dla średniej, $r = -0.83$ dla odchylenia standardowego) — im większy tramwaj, tym mniej przejazdów wystarcza do obsłużenia tego samego popytu. Podobnie R (czas obrotu) koreluje ujemnie ze średnią liczbą kursów ($r = -0.41$): wolniej wracający na pętlę pojazd fizycznie ogranicza liczbę możliwych kursów w ciągu godziny, co algorytm uwzględnia redukując częstotliwość.

Wagi funkcji celu działają zgodnie z założeniami modelu. Wzrost α — czyli mniejsza tolerancja na czekanie pasażerów na przystankach — wiąże się z dodatnią korelacją ze średnim minimum kursów ($r = 0.61$) i słabą dodatnią ze średnią ($r = 0.25$): system „dokłada”, pojazdy, by skrócić kolejki. Natomiast wzrost β , penalizujący dużą flotę na linii, koreluje ujemnie z liczbą kursów ($r = -0.32$ dla średniej, $r = -0.40$ dla minimum) — im wyższa kara kosztowa, tym algorytm agresywniej ogranicza liczbę pojazdów w ruchu.

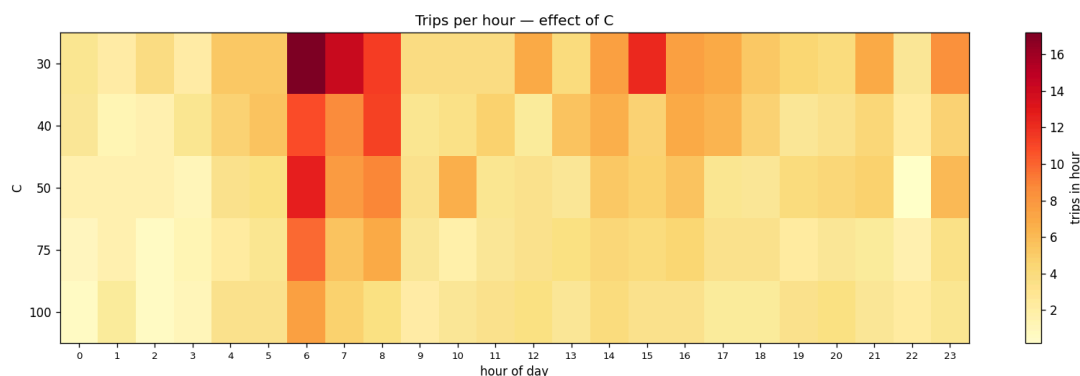


Rysunek 10: Zależność parametrów od średniej liczby kursów na godzinę z linią trendu i współczynnikiem r .

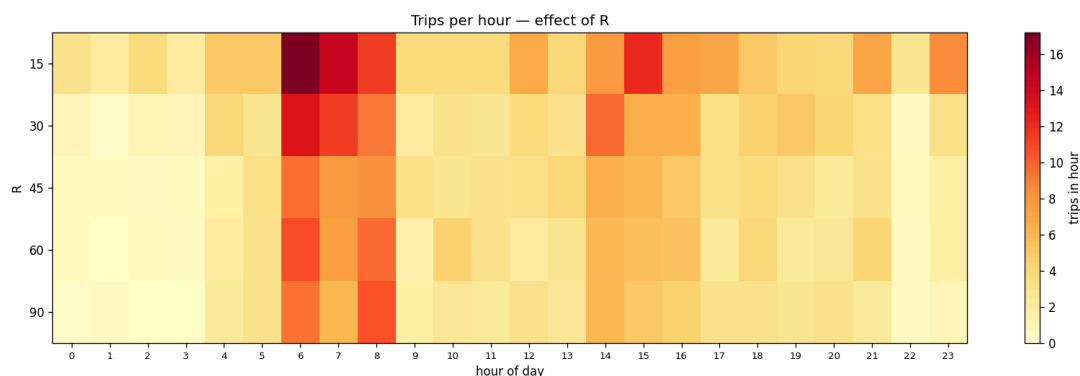
Wykresy rozrzutu z liniami trendu potwierdzają te obserwacje w sposób bardziej czytelny. Dla C widać wyraźny, niemal liniowy spadek średniej liczby kursów ($r = -0.65$): przejście z pojemności 30 do 100 pasażerów redukuje wymaganą częstotliwość o kilka kursów na godzinę. Dla R trend jest umiarkowany ($r = -0.39$) — dłuższy czas obrotu stopniowo obniża średnią liczbę kursów, choć wpływ ten jest słabszy niż pojemności. Dla α obserwujemy słaby, lecz konsekwentny wzrost ($r = 0.26$): od konfiguracji oszczędnej ($\alpha = 0.1$) do pasażerskiej ($\alpha = 20$) średnia liczba kursów rośnie. Dla β trend jest ujemny ($r = -0.31$): przy niskiej karze za flotę ($\beta = 5$) system wypuszcza zauważalnie więcej pojazdów niż przy wysokiej ($\beta = 100$).

5.3.2. Mapy ciepłe rozkładu kursów w ciągu doby

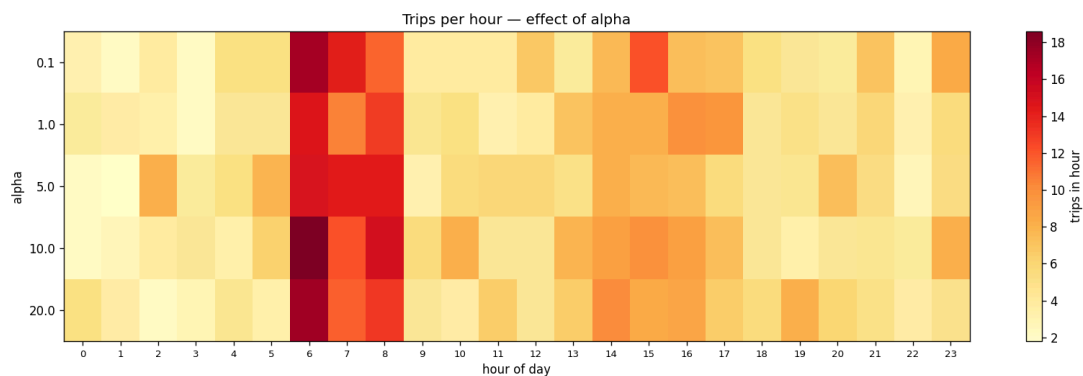
Poniższe mapy ciepłe przedstawiają, jak zmiana każdego z czterech parametrów przekłada się na liczbę kursów w poszczególnych godzinach doby (oś pozioma: godzina 0–23, oś pionowa: wartość parametru).



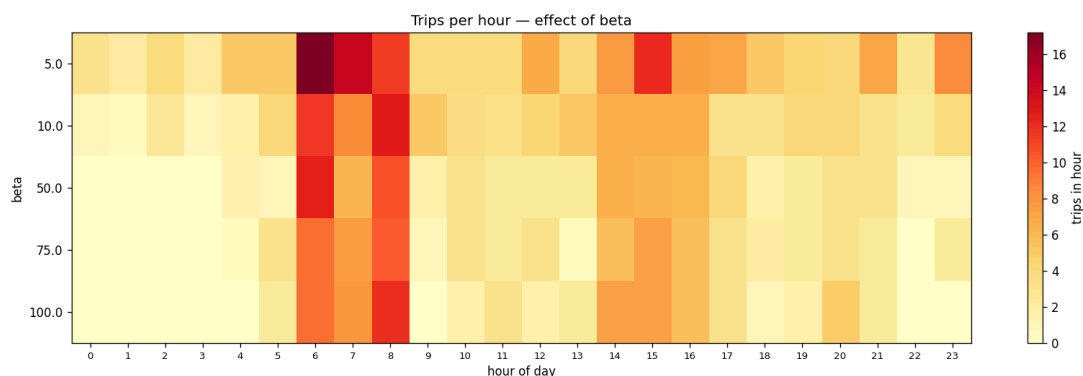
Rysunek 11: Wpływ pojemności C na liczbę kursów w poszczególnych godzinach.



Rysunek 12: Wpływ czasu obrotu R na liczbę kursów w poszczególnych godzinach.



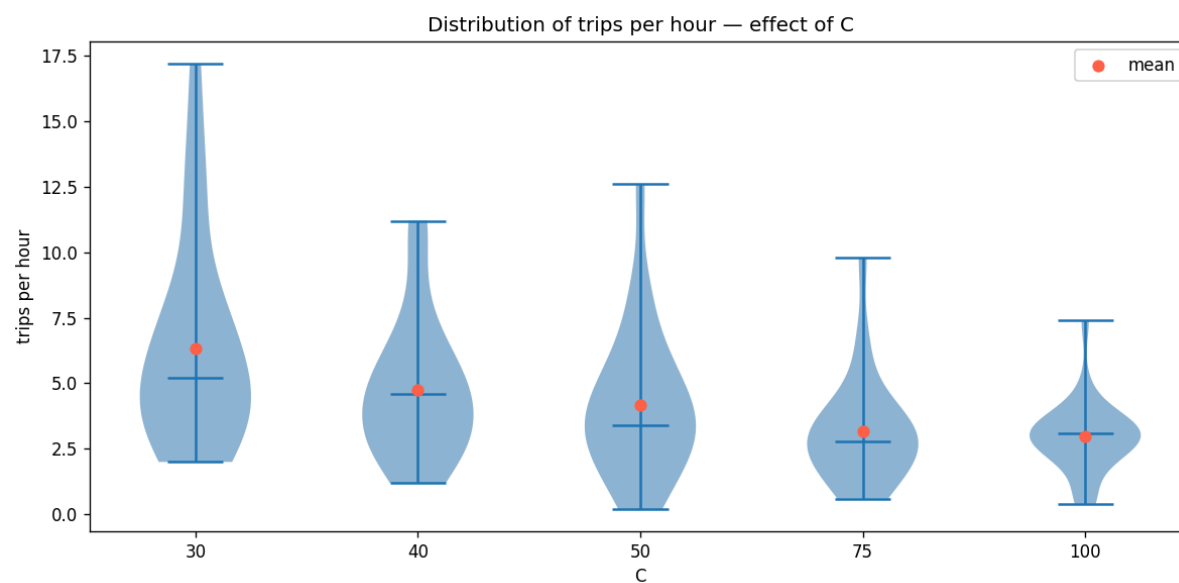
Rysunek 13: Wpływ wagi α na liczbę kursów w poszczególnych godzinach.



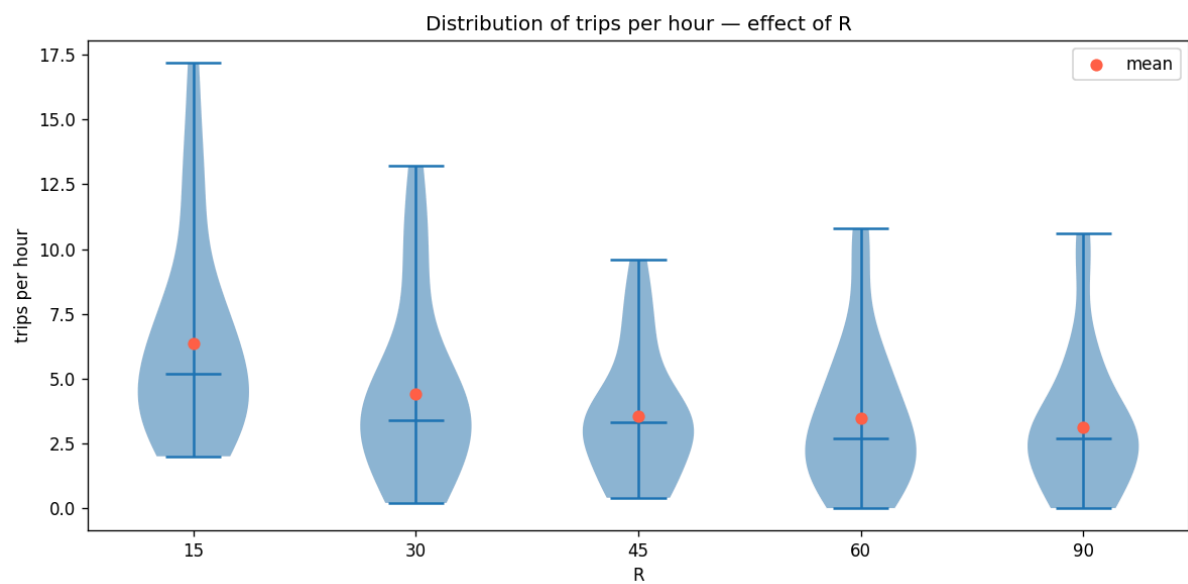
Rysunek 14: Wpływ wagi β na liczbę kursów w poszczególnych godzinach.

5.3.3. Rozkłady liczby kursów — wykresy skrzypcowe

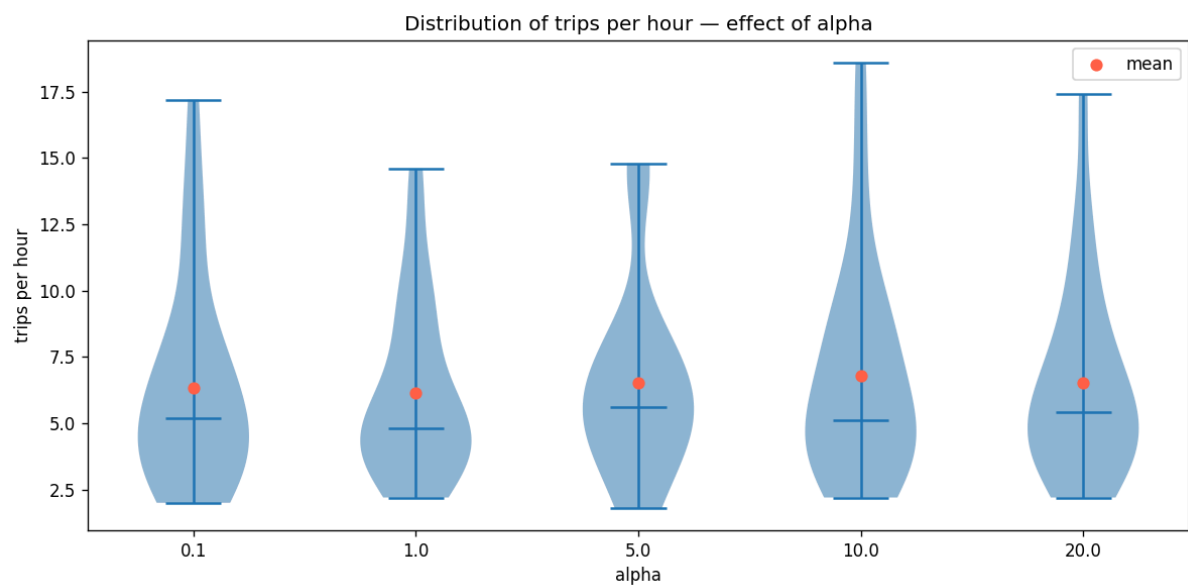
Wykresy skrzypcowe (violin) pokazują rozkład liczby kursów na godzinę dla kolejnych wartości każdego parametru. Czerwona kropka oznacza średnią, a szerokość „skrzypce,” — rozpiętość wyników między godzinami doby.



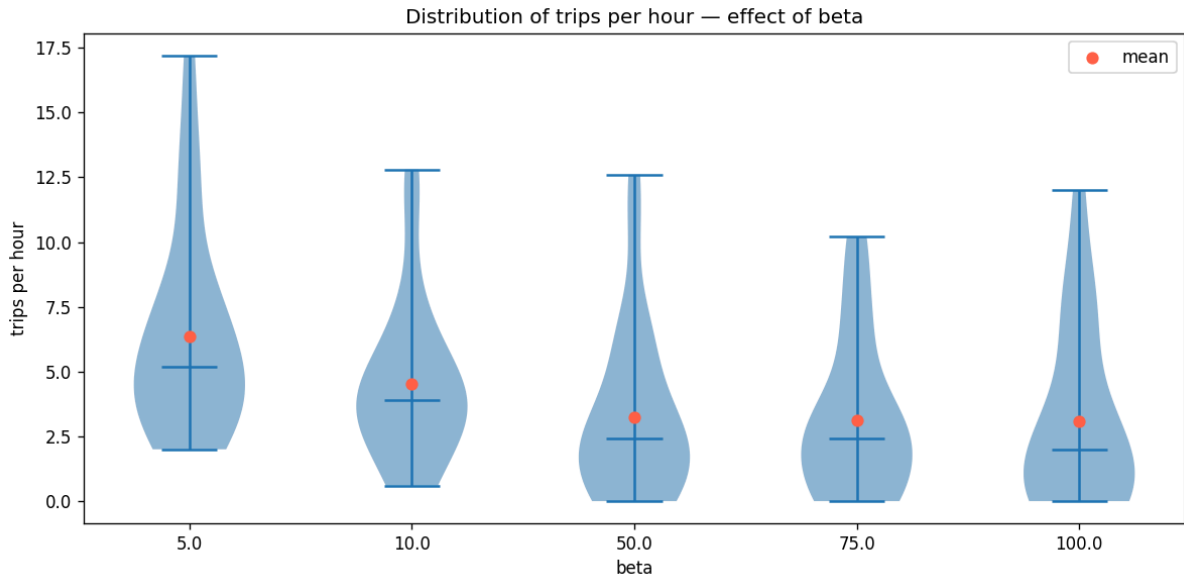
Rysunek 15: Rozkład kursów na godzinę w zależności od C .



Rysunek 16: Rozkład kursów na godzinę w zależności od R .



Rysunek 17: Rozkład kursów na godzinę w zależności od α .



Rysunek 18: Rozkład kursów na godzinę w zależności od β .

Podsumowując wyniki analizy parametrów:

- **C (pojemność):** większy tramwaj oznacza mniej potrzebnych przejazdów — efekt widoczny zarówno w godzinach szczytu, jak i poza nim.
- **R (czas obrotu):** wolniej wracający pojazd ogranicza liczbę możliwych kursów; wpływ rośnie wraz z wydłużaniem cyklu.
- **α (tolerancja oczekiwania):** im mniej akceptujemy kolejek na przystankach, tym więcej tramwajów algorytm wypuszcza w ruch.
- **β (kara za flotę):** im silniej penalizujemy dużą liczbę pojazdów na linii, tym rzadszy staje się rozkład jazdy.

6. Podsumowanie

W ramach projektu zaprojektowano i zaimplementowano kompletny system optymalizacji rozkładu jazdy tramwajów, łączący metody heurystyczne z ciągłym mikro-harmonogramowaniem. Dwufazowa architektura rozwiązania — Symulowane Wyżarzanie do wyznaczania częstotliwości godzinowych oraz algorytm dystrybucyjny CDF do generowania minutowych odjazdów — skutecznie adresuje kluczowe wyzwania planowania transportu miejskiego: dopasowanie podaży do zmiennego popytu, kompromis między komfortem pasażerów a kosztami operacyjnymi oraz eliminację niefizycznych luk czasowych na granicach okien godzinowych.

Główne osiągnięcia projektu obejmują:

- Sformułowanie modelu matematycznego z ważoną funkcją celu uwzględniającą czas oczekiwania i wielkość floty, z ograniczeniem przepustowości infrastruktury.
- Implementację metaheurystyki SA z deterministyczną inicjalizacją opartą na profilu popytu oraz mechanizmem śledzenia historycznie najlepszego rozwiązania.
- Opracowanie algorytmu Fazy 2 opartego na interpolacji ciągłej dystrybucyjny, który naturalnie wygładza przejścia między godzinami o różnym natężeniu ruchu.
- Stworzenie interaktywnej aplikacji (`app.ipynb`) z automatycznym generowaniem plakatów przystankowych w formacie PDF.
- Przeprowadzenie eksperymentów potwierdzających wpływ hiperparametrów wyżarzania oraz parametrów modelu (C , R , α , β) na jakość i kształt końcowego harmonogramu.

Eksperymenty numeryczne wykazały, że algorytm stabilnie zbiega do rozwiązań o energii $Z \approx 1119$ na standardowej instancji 24-godzinnej. Strojenie hiperparametrów SA potwierdziło dominujący wpływ `cooling_rate` (optymalnie 0.98–0.999), odporność na `min_temp` poniżej 10^{-2} oraz konieczność unikania ekstremalnie wysokiej `initial_temp`. Trendy te utrzymują się na instancji tygodniowej (168 h). Analiza wrażliwości parametrów modelu (625-kombinatoryczna siatka z 5 powtórzeniami) potwierdziła zgodność z intuicją operacyjną: większa pojemność taboru i dłuższy czas obrotu redukują wymaganą liczbę kursów, podczas gdy wzrost α zwiększa podaż, a wzrost β ją ogranicza.

Ograniczenia obecnej wersji systemu obejmują: pracę na syntetycznych danych testowych (brak walidacji na rzeczywistych danych ZTM), uproszczony model popytu bez uwzględnienia przesiadek i zmiennych warunków drogowych, oraz brak gwarancji optimum globalnego ze względu na charakter metaheurystyki. Kierunki dalszego rozwoju to integracja z rzeczywistymi danymi telemetrycznymi, rozszerzenie modelu o ograniczenia zajętości torowisk na węzłach oraz porównanie z rozwiązaniami dokładnymi (np. programowanie liniowe całkowitoliczbowe z modułu `solver.py`).

7. Dodatek – podział pracy

Etap projektu	Członek zespołu	Wkład (%)
Analiza i sformułowanie problemu	Grzegorz Grudzień	25
	Jan Martowicz	25
	Oliwia Zawila	25
	Maciej Borowiec	25
Opracowanie modelu matematycznego	Grzegorz Grudzień	25
	Jan Martowicz	25
	Oliwia Zawila	25
	Maciej Borowiec	25
Implementacja Fazy 1 (Symulowane Wyżarzanie)	Grzegorz Grudzień	100
	Jan Martowicz	0
	Oliwia Zawila	0
	Maciej Borowiec	0
Implementacja Fazy 2 (Mikro-harmonogramowanie)	Grzegorz Grudzień	0
	Jan Martowicz	100
	Oliwia Zawila	0
	Maciej Borowiec	0
Opracowanie modułu danych wejściowych	Grzegorz Grudzień	20
	Jan Martowicz	30
	Oliwia Zawila	30
	Maciej Borowiec	20
Opracowanie modułu danych wyjściowych	Grzegorz Grudzień	5
	Jan Martowicz	70
	Oliwia Zawila	5
	Maciej Borowiec	20

Integracja Systemu (Jupyter Notebook)	Grzegorz Grudzień	40
	Jan Martowicz	40
	Oliwia Zawila	10
	Maciej Borowiec	10
Eksperymenty, analiza parametrów modelu	Grzegorz Grudzień	60
	Jan Martowicz	0
	Oliwia Zawila	20
	Maciej Borowiec	20
Opracowanie raportu końcowego	Grzegorz Grudzień	0
	Jan Martowicz	10
	Oliwia Zawila	50
	Maciej Borowiec	40